"Registration: " – note that when operating in this way, strstr() does not return the search sub-string in the result.

The corollary to the functions we've just seen is strrchar(), which functions in all ways as strstr() does except that it searches for the last occurrence of a sub-string:

```
$ships = 'NCC-1701-B NCC-1701-C NCC-1701-D';
print strrchr($ships, 'NCC') . PHP_EOL;
print strrchr($ships, 'NCC', true) . PHP_EOL;
```

```
Output:
NCC-1701-D
NCC-1701-B NCC-1701-C
```

Remember when I told you that there were inconsistencies with the conventions of PHP core string functions? Here's one of them: there is no case-insensitive form of strrchr(). There is, however, a case-insensitive form of the mbstring equivalent, which you can use quite safely on ASCII-encoded data (though it comes with a slight performance overhead). Speaking of mbstring…

mb_strstr(), mb_stristr(), mb_strrchr(), mb_strrichr()

There are multi-byte safe equivalents for strstr(), stristr(), and strrchr(). There's even an mb_strrichr() function even though there's no "strrichr" equivalent (no, I have no idea why). All of them have the same function signature as their core counterparts but will take an additional string to specify the encoding (otherwise the default encoding will be used. Let's take another look at the problematic Polish UTF-8 character from an earlier example.

```
$name = "Name: \u{141}ukasz";
print mb_stristr($name, "\u{142}");
```

```
Output:
Łukasz
```

By replacing stristr() with mb_stristr(), the full Unicode mapping of upper-and-lowercase characters is made available, and we get the expected result. Here's the same thing again, but this time using raw byte values for UTF-16 encoding:

```
//            N       a       m       e       :       <SPC>
$utf16name = "\x00\x4e\x00\x61\x00\x6d\x00\x65\x00\x3a\x00\x20"
```